

SRIRAM VIVEK

Charlotte, NC | +1 (631) 428-3788 | sriramv1202@gmail.com | linkedin.com/in/sriram-vivek | [Portfolio](#) | github.com/SriramV1212

Experience

Galatea Associates LLC

Financial Software Engineer Intern

Boston, Massachusetts

June 2025 – August 2025

- Contributed to an AI proof-of-concept for a major financial services client designed to help financial advisors interpret portfolio optimizer trade suggestions, by building a RAG service that supplied the LLM with retrievable domain context for narrative generation
- Built a document ingestion pipeline extracting text and tables from a 600-page optimizer specification PDF using PyMuPDF, chunking content with LangChain's Recursive Character Text Splitter, generating embeddings via text-embedding-3-large, and storing vectors in a PGVector store for similarity retrieval
- Investigated gaps in the ingested specification and validated that retrieval surfaced usable context to inform the main narrative prompt, working through an unfamiliar production codebase with limited direction
- Wrote SQL queries against MS SQL Server to extract and reshape portfolio positions, constraints, and trade outputs into structured tables for LLM prompt consumption, and authored technical documentation for the RAG service and the broader POC in Confluence while tracking deliverables in JIRA under Agile methodology, collaborating closely with the broader engineering team throughout

Projects

Agentic RAG System with Custom MCP Server | [GitHub](#)

- Built a RAG backend over Anthropic API documentation, designing a Markdown-header-aware chunking pipeline with a token-bounded fallback and ingesting 3,100+ document chunks into a self-hosted Qdrant vector store for semantic retrieval
- Built a custom Python MCP server exposing 4 retrieval tools as the agent's Qdrant access path, keeping direct vector-store access outside the agent and verifying all 4 tools with a standalone MCP test client
- Deployed the backend to a self-managed Linux VPS, running Qdrant and OpenClaw with Docker, FastAPI and MCP as systemd services, and Nginx/Certbot for HTTPS; built a self-hosted GitHub Actions workflow to sync dependencies, recreate secrets, ensure containers are running, and restart backend services on pushes to main
- Built and deployed a Next.js/TypeScript frontend on Vercel with a retrieval inspector showing retrieved chunks, similarity scores, headings, and source URLs alongside grounded answers and explicit abstentions

Distributed Microservices Orchestration using gRPC | [GitHub](#)

- Designed a 3-service gRPC backend with Protocol Buffers for a flight-booking workflow, where an Orchestrator coordinates User and Search services; implemented unary RPCs for booking and a separate server-side streaming RPC for live flight-price updates
- Added exponential-backoff retries around both downstream calls and a circuit breaker around the Search call path, and secured Orchestrator-to-User and Orchestrator-to-Search connections with mTLS requiring client authentication
- Instrumented all 3 services with OpenTelemetry tracing and Prometheus request, error, and latency metrics, and ran Jaeger, Prometheus, and Grafana via Docker Compose for trace inspection and dashboards
- Validated retry and circuit-breaker behavior using client flows for standard booking and server-side streaming, along with a manually enabled 50% Search-service failure simulation

Real-Time Event-Driven Payment Processing Backend | [GitHub](#)

- Architected an event-driven payment backend with FastAPI, Apache Kafka, and PostgreSQL, separating synchronous payment creation from asynchronous consumer processing and tracking payments across pending, processed, and failed states
- Built idempotent consumer processing using a processed-events table with conflict-safe event-ID inserts, disabling Kafka auto-commit and committing offsets only after database success or DLQ handoff so redelivered events can be handled safely after consumer failures
- Published failed events to a dedicated dead-letter queue with the original event, error, timestamp, partition, and offset, and load-tested the end-to-end API-to-consumer path with 1,000 simulated payment requests on a 4-partition Kafka topic
- Configured a 4-partition Kafka topic with messages keyed by user ID and a shared consumer group, allowing multiple consumer processes to handle partitions in parallel while routing events for the same user consistently to the same partition

Education

Stony Brook University

Master of Science in Computer Science and Applied Mathematics (Data Science)

New York, United States

Aug 2024 – May 2026

Courses: Data Structures and Algorithms, Computer Networks, Programming Abstractions, Theory of Computation, Data Management, Big Data Systems, Big Data Analysis, Probability, Data Analysis, Statistical Learning, Statistical Computing

SSN College of Engineering

Bachelor of Engineering in Electrical and Electronics Engineering

Chennai, India

Nov 2020 – May 2024

Courses: Python Programming, Object-Oriented Programming, Linear Algebra and Calculus, Partial Differential Equations

Skills

Programming Languages: Python, TypeScript, SQL, Linux

Backend & Distributed Systems: FastAPI, gRPC, Apache Kafka, Apache Spark, PostgreSQL, MySQL, MS SQL Server, MongoDB, Redis

Frontend: Next.js, TypeScript, Vercel

Cloud & Deployment: AWS (S3, EC2, Lambda), Azure, Docker, Kubernetes, GitHub Actions, Git, Nginx, systemd

Observability & Telemetry: Prometheus, Grafana, Jaeger, OpenTelemetry, distributed tracing

Generative AI & LLMs: RAG, LangChain, LangGraph, Model Context Protocol (MCP), Prompt Engineering, OpenClaw, Hermes, Pinecone, ChromaDB, Qdrant

AI-Assisted Development: Claude Code CLI, Codex, Cursor